Stability

The main limitation of digital filters carried out by convolution is *execution time*. It is possible to achieve nearly any filter response, provided you are willing to wait for the result. Recursive filters are just the opposite. They run like lightning; however, they are limited in performance. For example, consider a 6 pole, 0.5% ripple, low-pass filter with a 0.01 cutoff frequency. The recursion coefficients for this filter can be obtained from Table 20-1:

```
a0=  1.391351E-10
a1=  8.348109E-10  b1=  5.883343E+00
a2=  2.087027E-09  b2= -1.442798E+01
a3=  2.782703E-09  b3=  1.887786E+01
a4=  2.087027E-09  b4= -1.389914E+01
a5=  8.348109E-10  b5=  5.459909E+00
a6=  1.391351E-10  b6= -8.939932E-01
```

Look carefully at these coefficients. The "b" coefficients have an absolute value of about *ten*. Using single precision, the round-off noise on each of these numbers is about one ten-millionth of the value, i.e., 10^{-6}. Now look at the "a" coefficients, with a value of about 10^{-9}. Something is obviously wrong here. The contribution from the input signal (via the "a" coefficients) will be 1000 times smaller than the *noise* from the previously calculated output signal (via the "b" coefficients). This filter won't work! In short, round-off noise limits the number of poles that can be used in a filter. The actual number will depend slightly on the ripple and if it is a high or low-pass filter. The approximate numbers for single precision are:

TABLE 20-3
The maximum number of poles for single precision.

Cutoff frequency	0.02	0.05	0.10	0.25	0.40	0.45	0.48
Maximum poles	4	6	10	20	10	6	4

The filter's performance will start to degrade as this limit is approached; the step response will show more overshoot, the stopband attenuation will be poor, and the frequency response will have excessive ripple. If the filter is pushed too far, or there is an error in the coefficients, the output will probably oscillate until an overflow occurs.

There are two ways of extending the maximum number of poles that can be used. First, use double precision. This requires using double precision in the coefficient calculation as well (including the value for *pi*).

The second method is to implement the filter in *stages*. For example, a six pole filter starts out as a cascade of three stages of two poles each. The program in Table 20-4 combines these three stages into a single set of recursion coefficients for easier programming. However, the filter is more stable if carried out as the original three separate stages. This requires knowing the "a" and "b" coefficients for each of the stages. These can